EfficientUICoder: A Bidirectional Token Compression Framework for Efficient MLLM-Based UI Code Generation

JINGYU XIAO, The Chinese University of Hong Kong, China
ZHONGYI ZHANG, Huazhong University of Science and Technology, China
YUXUAN WAN, The Chinese University of Hong Kong, China
YINTONG HUO*, Singapore Management University, Singapore
YANG LIU, Nanyang Technological University, Singapore
MICHAEL R. LYU, The Chinese University of Hong Kong, China

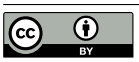
Multimodal Large Language Models (MLLMs) have demonstrated exceptional performance in UI2Code tasks (i.e., generating code from UI mockups), significantly enhancing website development efficiency. However, UI2Code tasks incur substantially higher computational overhead compared to traditional code generation tasks. This overhead is primarily driven by the large number of input image tokens required to represent complex visual designs and the extensive volume of output code tokens needed to describe complete webpage structures. In this paper, we conduct a comprehensive preliminary study on popular MLLMs for UI2Code tasks, identifying significant redundancies in both image and code tokens. We observe that these redundancies not only exacerbate computational complexity but also hinder the model's ability to focus on key UI elements, leading to excessively lengthy and often invalid HTML files. To address these challenges, we propose EfficientUICoder, a bidirectional compression framework designed for efficient UI code generation. First, we introduce an Element and Layout-aware Token Compression method, which preserves essential UI element and layout information by detecting element regions and constructing a UI element tree for efficient representation. Second, we design a Region-aware Token Refinement strategy that refines selected tokens by leveraging attention scores to evaluate semantic importance, discarding low-attention tokens from selected regions while integrating high-attention tokens from unselected regions. Third, we develop an Adaptive Duplicate Token Suppression mechanism, which dynamically modulates token probabilities during decoding by tracking HTML/CSS code structure frequencies and applying exponential penalty strategies to minimize repetitive generation. Extensive experiments demonstrate that EfficientUICoder achieves a **55%-60%** compression ratio without compromising the quality of the generated webpages, effectively reducing output code redundancy. In terms of efficiency, EfficientUICoder achieves superior improvements, reducing computational cost by up to **44.9%**, generated tokens by up to **41.4%**, prefill time by up to **46.6%**, and inference time by up to **48.8%** on 34B-level MLLMs. Code is available at https://github.com/WebPAI/EfficientUICoder.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Multi-modal Large Language Model, Code Generation, Token Compression, Web Development

*Yintong Huo is the corresponding author.

Authors' Contact Information: Jingyu Xiao, The Chinese University of Hong Kong, Hong Kong, China, jyxiao@link.cuhk.edu.hk; Zhongyi Zhang, Huazhong University of Science and Technology, Wuhan, China, zyzhang1@hust.edu.cn; Yuxuan Wan, The Chinese University of Hong Kong, Hong Kong, China, yxwan9@cse.cuhk.edu.hk; Yintong Huo, Singapore Management University, Singapore, Singapore, ythuo@smu.edu.sg; Yang Liu, Nanyang Technological University, Singapore, Singapore, yangliu@ntu.edu.sg; Michael R. Lyu, The Chinese University of Hong Kong, Hong Kong, China, lyu@cse.cuhk.edu.hk.



ACM 2994-970X/2026/7-ARTFSE107
https://doi.org/10.1145/3808114

ACM Reference Format:
Jingyu Xiao, Zhongyi Zhang, Yuxuan Wan, Yintong Huo, Yang Liu, and Michael R. Lyu. 2026. EfficientUICoder: A Bidirectional Token Compression Framework for Efficient MLLM-Based UI Code Generation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE107 (July 2026), 23 pages. https://doi.org/10.1145/3808114

1 Introduction

Converting webpage designs into functional UI code represents a critical yet labor-intensive bottleneck in web development. Manual translation of UI designs into code is both time-consuming and requires specialized domain expertise, creating significant barriers to rapid web development. Multimodal Large Language Models (MLLMs), with their remarkable capabilities in visual-language understanding, show significant potential for visually rich code generation applications, such as creating UIs [30, 42], slides [27], and posters [21]. This success has recently spurred considerable interest within the software engineering research community, particularly in UI-to-code (UI2Code) conversion. For example, DCGen [30] employs a divide-and-conquer strategy to generate submodule code before assembly, while DeclarUI [44] combines element segmentation with page transition graphs for mobile app UI generation. UICopilot [11] adopts hierarchical generation by creating HTML structures before components, and LayoutCoder [32] introduces layout-aware frameworks for preserving UI layout fidelity. However, none of them consider the computational overhead and efficiency of UI2Code tasks.

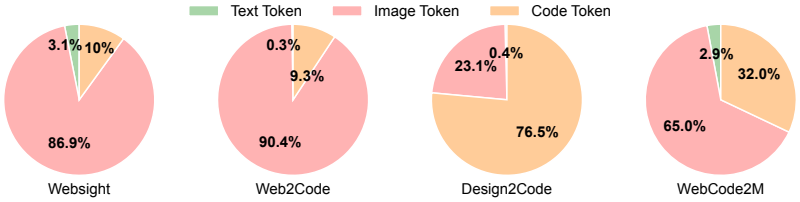


Fig. 1. The token ratios of different datasets.

Compared to traditional code generation tasks [20, 24, 26, 39], UI2Code presents unique computational challenges. These tasks consume substantially more tokens due to two primary factors: the extensive number of input image tokens required to represent complex visual designs, and the large volume of generated code tokens needed to describe complete webpage structures. We analyze the token distribution across several mainstream UI2Code benchmarks, including WebSight [15], Web2code [41], Design2Code [25], and WebCode2M [9]. As illustrated in Fig. 1, the combined image token and code token ratio accounts for more than 90% of the total token consumption.

These excessively long sequences of image and code tokens consume substantial memory and computational resources throughout the entire MLLM pipeline during code generation. On the one hand, previous studies [6] have demonstrated that the information contained in images is much sparser than that in text. On the other hand, several studies [12, 25] have pointed out that code generated by LLMs frequently includes redundant information. Hence, a natural question arises: **"Are all image and code tokens necessary for UI code generation?"**

To identify redundancy in image tokens (Section 3.1), we conduct an in-depth analysis of visual tokens generated by the widely adopted vision encoder CLIP [23]. Attention score visualization reveals that redundant visual tokens not only inflate computational costs but also misdirect attention toward uninformative regions (e.g., background areas), thereby diminishing the model's focus on critical UI elements. To examine code redundancy (Section 3.2), we evaluate two prominent UI2Code benchmarks Design2Code and WebCode2M using popular open-source MLLMs (Llava-v1.6-7b and

Llava-v1.6-34b [18]). Through systematic analysis of the outputs, we identify three redundancy types, demonstrating that MLLMs frequently produce superfluous HTML/CSS structures and textual content. This code redundancy not only increases computational overhead but also entraps models in cyclical generation patterns, potentially resulting in invalid HTML structures.

Based on these findings, we propose the first multimodal token compression framework for UI-to-code, namely EfficientUICoder. EfficientUICoder mitigates redundant image tokens during the encoding stage and suppresses redundant code tokens during the decoding stage. First, we propose an **Element and Layout-aware Token Compression (ELTC)** method to compress redundant tokens while preserving UI element and layout information. It initially employs a UI element detection algorithm to identify element regions and constructs a UI element graph to model the UI layout structure. Subsequently, a minimum spanning tree algorithm is applied to obtain the most efficient UI representation in the form of a UI element tree. Second, we devise a **Region-aware Token Refinement (RTR)** strategy to refine the tokens selected in the first stage. It leverages attention scores to evaluate the semantic importance of visual tokens and removes tokens with lower attention scores that were selected by the first stage. Recognizing that certain critical tokens may reside in background regions, it selectively incorporates high-attention tokens from these areas. By carefully balancing the dropping ratio and selection ratio, we achieve substantial token compression while preserving the most semantically important visual information across both foreground and background regions. Third, we design an **Adaptive Duplicate Token Suppression (ADTS)** method to modulate token probabilities during the decoding stage. It first employs HTML and CSS parsers to track the frequencies of HTML/CSS code structures and textual content. Then an exponential penalty strategy is applied to adjust the probabilities of subsequent tokens, thereby suppressing repetitive generation based on the frequency of repetitions. We conduct experiments on widely used UI2Code benchmark Design2Code and WebCode2M using Llava-v1.6 series models. Extensive experiments demonstrate that EfficientUICoder achieves a 55%-60% compression ratio without compromising UI2Code performance, effectively reducing output code redundancy. In terms of efficiency, EfficientUICoder achieves superior improvements, reducing computational cost by up to 44.9%, generated tokens by up to 41.4%, prefill time by up to 46.6%, and inference time by up to 48.8% on 34B-level MLLMs. Our contributions are as follows:

- We conduct an in-depth analysis of token redundancy in UI2Code tasks, identifying two redundancy patterns in the typical UI-to-code generation process.
- We propose EfficientUICoder, the first multimodal bidirectional token compression framework that reduces visual redundancy by constructing a UI element tree to preserve key UI elements and layout structures during encoding, while mitigating code redundancy by decreasing the probabilities of repeated tokens during the decoding stage.
- We conduct extensive experiments across popular UI2Code benchmarks and state-of-the-art MLLMs, demonstrating that our approach achieves substantial token compression and efficiency while maintaining generated UI quality.

2 Background

2.1 UI-to-Code Task Definition

The UI2Code task involves translating visual webpage designs into functional HTML+CSS code that accurately reproduces the original design. Let I_0 denote the input design image of a webpage, and let C_0 represent the ground-truth HTML+CSS code. Given the design image I_0, a Multimodal Large Language Model M generates code $C_g = M(I_0)$, where the rendered output I_g produced by executing C_g should closely approximate I_0 in both structural layout and visual appearance.

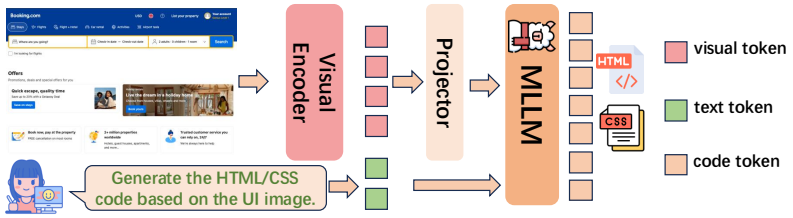


Fig. 2. MLLM-based UI2Code task pipeline.

2.2 MLLMs Background

Architecture. As illustrated in Fig. 2, MLLM architectures typically comprise three core components: a visual encoder, a modality projector, and a large language model (LLM). The visual encoder, commonly implemented as a pre-trained image encoder such as CLIP's vision model [23], first converts the image into patch blocks and then transforms input images into visual token representations. The projector module serves as a bridge that aligns these visual tokens with the LLM's word embedding space, thereby enabling the LLM to effectively process visual information. Subsequently, the LLM integrates the aligned visual and textual representations to generate coherent responses.

Computational Complexity. Assessing the computational complexity of MLLMs necessitates analyzing key architectural components, particularly the self-attention mechanism and the feed-forward network (FFN). The total floating-point operations (FLOPs) can be formulated as:

$$\text{Total FLOPs} = T \times \left(4nd^2 + 2n^2d + 2ndm\right), \tag{1}$$

where T denotes the number of transformer layers, n represents the sequence length, d indicates the hidden dimension size, and m corresponds to the intermediate size of the FFN. This formulation demonstrates that computational complexity exhibits a strong dependence on the sequence length n. In typical MLLM applications, the sequence length is defined as:

$$n = n_{\text{img}} + n_{\text{text}}, \tag{2}$$

where n_{img} frequently exceeds n_{text} by substantial margins, often by a factor of 20 or more. Thus, reducing n_{img} represents a critical strategy for enhancing the computational efficiency of MLLMs.

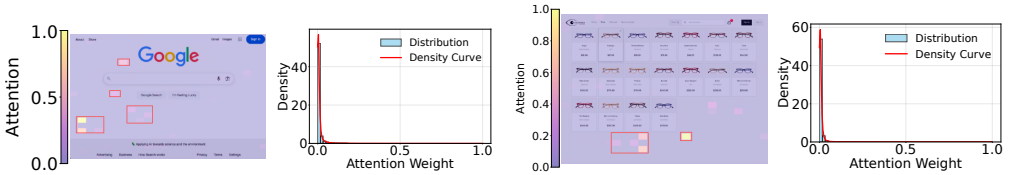
3 Preliminary Study

3.1 Visual Token Redundancy

In practical front-end development, developers don't need to examine every pixel to implement a webpage. Many visual components contain inherently redundant information, including image placeholder, uniform background regions and so on. For any given element, experienced programmers can infer critical specifications such as color, size, and styling properties from a subset of representative pixels rather than exhaustive full pixel visual analysis.

To investigate "*where are MLLMs looking at*" on the UI image, we conduct an in-depth analysis on the visual tokens generated by the widely used vision encoders, CLIP [23]. Fig. 3 shows the normalized attention score distribution on two webpages (i.e., search engine webpage and shopping webpage), we use red bounding boxes to mark the area with high attention and find that only a few tokens receive high attention, while most visual tokens receive minimal attention. Surprisingly, the models pay more attention to the empty background part of the UI image than to the UI elements.

Redundant visual tokens introduce two detrimental effects on model performance: (1) *Computational overhead escalation*: redundant visual tokens substantially increase the sequence length of inputs, leading to quadratic growth in attention computation complexity. (2) *Attention distraction*:



(a) Search webpage and attention distribution on it. (b) Shopping webpage and attention distribution on it.

Fig. 3. Visual encoders' attention score visualization and distribution on two webpages.

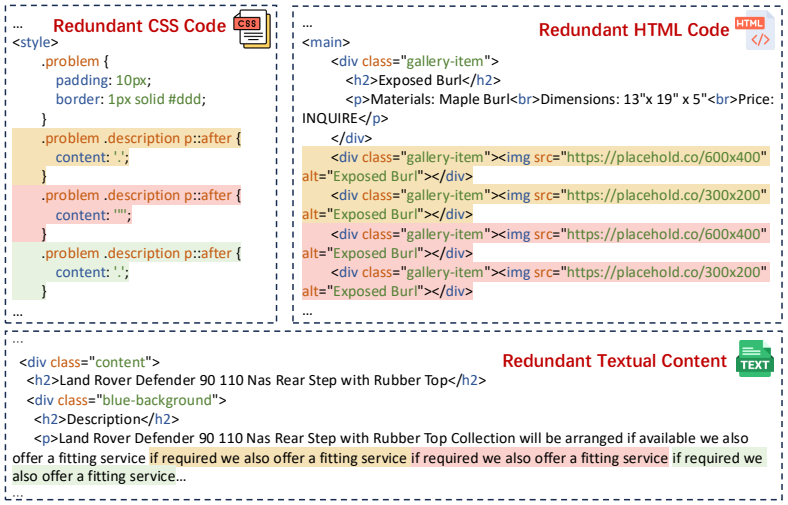


Fig. 4. Duplicate code token examples.

excessive redundant tokens (e.g., background) scatter the model's attention across uninformative visual regions, diminishing its capacity to focus on semantically crucial elements.

Observation 1: Redundant visual tokens inflate computational cost and divert attention to uninformative regions (e.g., background), reducing the model's focus on critical UI elements.

3.2 Code Token Redundancy

We conduct experiments on Llava-v1.6 (7B and 34B) [18] to study their output code on UI2Code task. We select the widely used Design2Code [25] dataset and adopt the "direct prompt" in design2code [25] for evaluation. In our experiments, we deploy Multi-modal Large Language Models (MLLMs) on four NVIDIA RTX A800 GPUs, each with 80G memory. The maximum generated code length is set to 4096. And we set the temperatures as 0 for getting stable output.

We engage a PhD student with extensive front-end development expertise to manually screen out samples with redundant content and then conduct analysis. Our systematic analysis reveals three primary categories of redundancy issues: (1) *Redundant CSS code*: As demonstrated in the upper-left panel of Fig. 4, identical CSS selectors such as ".problem .description p::after" are generated multiple times with duplicate styling properties. (2) *Redundant HTML Code*: The upper-right panel illustrates unnecessary repetition of structural patterns, where multiple " " elements

Table 1. The redundant statistics on two datasets. "Endless" denotes the repetition continue until the model reaches its maximum token limit. "End" means that the repetition terminates after repeated several times.

Model	Dataset	Redundant CSS Code		Redundant HTML Code		Redundant Textual Content		Total
		End	Endless	End	Endless	End	Endless	
Llava-v1.6-7b	Design2Code	37	10	16	54	4	27	148
	WebCode2M	7	1	5	19	0	7	39
Llava-v1.6-34b	Design2Code	0	46	3	54	0	33	139
	WebCode2M	0	5	1	12	0	15	33

with identical content and attributes are generated redundantly. (3) *Redundant textual content*: The bottom panel exemplifies duplicate text generation, where phrases such as "we also offer a fitting service if required" are repeated multiple times within the same content block.

This repetitive pattern may continue indefinitely until the model reaches its maximum token limit. We calculate the ratio of different issues, with results presented in Table 1. Analysis of the rendered outputs reveals that such redundant generation introduces several detrimental effects: it inflates token consumption, increases webpage length, and extends generation time. More critically, excessive repetition can trap the model in generating a particular element, causing it to ignore subsequent elements or fail entirely to produce valid HTML structures.

Observation 2: MLLMs frequently produce redundant HTML/CSS structures, and textual content, which not only increases computational overhead but also trap models in cyclical generation patterns, even leading to invalid html structures.

4 Methodology

Overview. Based on the two observations, we propose EfficientUICoder (as shown in Fig. 5) to mitigate the image and code token redundancy. EfficientUICoder first applies the Element and Layout-aware Token Compression module (Section 4.1) to get the efficient UI representation by constructing a UI element tree. Then the Region-aware Token Refinement module (Section 4.2) refines selected tokens by leveraging attention scores to evaluate semantic importance, discarding low-attention tokens in the selected regions while integrating high-attention tokens from unselected regions. Finally, the Adaptive Duplicate Token Suppression strategy (Section 4.3) first identifies the code redundancy by tracking the HTML/CSS code structure and text frequencies, then dynamically penalizes token probabilities during decoding.

4.1 Element and Layout-Aware Token Compression

We aim to represent UI designs with minimal pixel while preserving semantic accuracy. This objective encompasses two critical requirements: ensuring no UI elements are omitted and maintaining correct spatial relationships between different UI components. To achieve this goal, we propose an Element and Layout-aware Token Compression method that constructs efficient UI representations by building a UI element tree structure that explicitly considers the underlying UI layout hierarchy.

4.1.1 UI Element Tree Definition. We first define the concept of a *UI Element Graph* $G = (V, E)$ as a concise representation of user interfaces, where each node $v_i \in V$ corresponds to a distinct UI element region within the interface, including textual content, images, and components (e.g., button, input box). For any two nodes v_i and v_j with bounding boxes B_i and B_j respectively, the edge $e_{ij} \in E$ is the shortest link between two bounding boxes, the weight w_{ij} represents the spatial

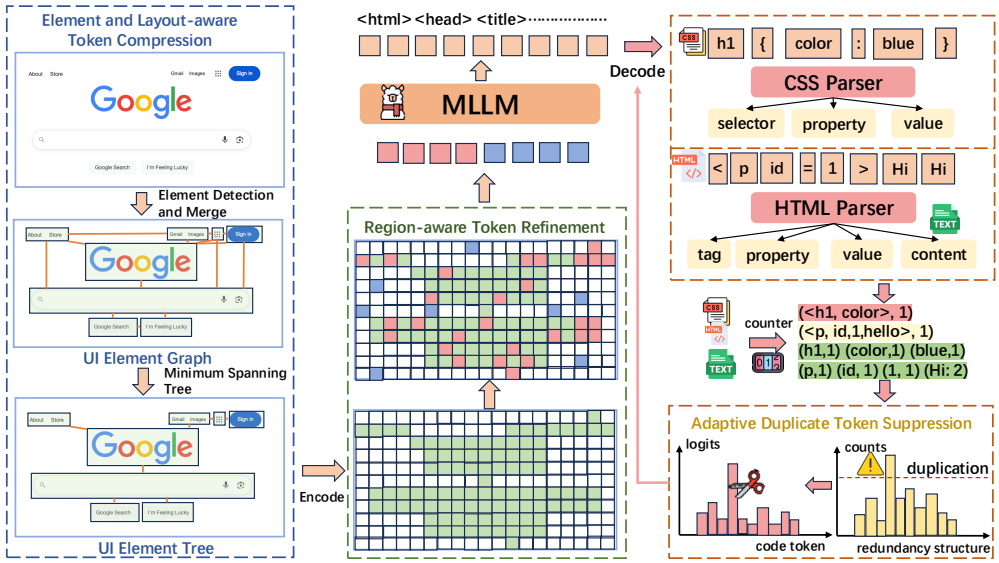


Fig. 5. EfficientUICoder framework.

relationship defined by the minimum distance between the boundaries of the two bounding boxes:

$$w_{ij} = \min_{p \in \partial B_i, q \in \partial B_j} \|p - q\|_2, \tag{3}$$

where ∂B_i and ∂B_j denote the boundaries of bounding boxes B_i and B_j, and $\|\cdot\|_2$ represents the Euclidean norm, p and q are the points on the ∂B_i and ∂B_j. Upon this, *UI Element Tree* $T = (V, E')$ is defined as a representation that consumes the least token, where $E' \subset E$, $|E'| = |V| - 1$ and T is acyclic and connected. The problem is formulated as:

$$T^* = \arg\min_{T=(V,E')} \sum_{(v_i,v_j)\in E'} w_{ij}. \tag{4}$$

4.1.2 UI Element Tree Construction. First, EfficientUICoder employs the UI Element Detection (UIED) algorithm [36] to extract bounding boxes for all elements (i.e., text, image and component) from the UI image. Due to inter-word spacing, UIED fragments text elements into numerous small segments, a continuous sentence may be split into multiple parts. To merge these fragments, we establish a distance threshold for each text element: if the distances between text bounding boxes fall below this threshold, the fragments are merged. In the experiments, the threshold is set to 50 pixels to achieve better merging performance. Furthermore, to ensure non-overlapping bounding boxes, we first classify the overlap relationship between boxes. In cases of inclusion, we retain the largest bounding box; for intersection relationships, we compute the minimum enclosing rectangle.

Second, EfficientUICoder calculates the shortest link e_{ij} between two element regions (i.e., bounding boxes) v_i and v_j, and computes the corresponding weight w_{ij} to construct the UI element Graph G. The shortest distance computation follows three cases: (1) If two bounding boxes are horizontally separated but vertically overlapping, the shortest line segment is horizontal; (2) If two bounding boxes are vertically separated but horizontally overlapping, the shortest line segment is vertical; (3) If two bounding boxes are separated in both dimensions, the shortest line segment connects the two closest corner points.

Finally, after the UI elememt graph G is construted, EfficientUICoder employs the Minimum Spanning Tree (MST) algorithm Kruskal [17] to construct a UI element tree T.

4.2 Region-Aware Token Refinement

Although substantial token compression is achieved in the first step, we observe that redundancy persists within element regions detected by UIED [36]. For instance, in Fig 5, the search input box component still contain redundant visual information, i.e., the tokens in the blank part of the search box. Similarly, some unselected background areas also contain some key information (e.g., background color), we need to add these important tokens. Therefore, we drop some unimportant tokens in the selected area and select some important tokens in the unselected area. This approach achieves substantial token compression while preserving the most semantically important visual information across both selected and unselected regions.

We assess token importance through attention score within the visual encoder. We compute attention scores as $A_{avg} = \frac{1}{H}\sum_{h=1}^{H} \text{Softmax}\left(\frac{Q_h K_h^\top}{\sqrt{D_h}}\right)$, where H is the attention head counts, D_h denotes the head dimension, and Q_h and K_h correspond to query and key matrices, respectively. Suppose the token number is N, by averaging across all attention heads, we obtain an aggregated attention matrix $A_{avg} \in \mathbb{R}^{\mathrm{N}\times\mathrm{N}}$, which captures the attention relationships between all token pairs.

For CLIP [23] visual encoder, the CLS token aggregates information from the entire image, so we use the CLS token's attention scores to assess token importance. The importance score of token i is $I_i = A_{avg}[\text{CLS}, i], \quad i = 1, 2, \ldots, N$, where I_i represents the attention weight from the CLS token to token i. Given the token sets S (selected tokens) and U (unselected tokens) from the first stage, we remove a proportion r of the least important tokens from set S and simultaneously select a proportion r of the most important tokens from set U.

4.3 Adaptive Duplicate Token Suppression

As shown in Section 3.2, there are three types of output redundancy: css code redundancy, html code redundancy and text content redundancy. These redundancies manifest as continuous patterns in the output sequence, so we can monitor repetition frequency during the decoding phase and subsequently suppress duplications by reducing the probability of repeated tokens. Although repetition suppression is implemented as a standard technique in popular inference libraries (e.g., Huggingface Transformers [31]), it cannot be applied in our setting. The standard repetition penalty applies a uniform penalty to all previously generated tokens, which is unsuitable for UI2Code scenarios. In particular, it will excessively suppress tokens that are expected to appear repeatedly, such as '`div`' and '`p`' tags, which naturally occur multiple times in webpages. Accordingly, we propose an Adaptive Duplicate Token Suppression method, which consists of a repetition frequency tracking process for recording the repetition counts and an exponential penalty strategy to dynamic adjust the probabilities of the repeated token.

4.3.1 Repetition Frequency Tracking. As shown in Algorithm 1, we implement separate frequency tracking for CSS, HTML, and text content, leveraging the distinct syntactic markers in web markup languages. Because length("<style>")=7 and length("<body>")=6, during frequency tracking, if last 7 characters of the generated sequence (seq[-7:]) are "<style>" (line 7), we begin the css code tracking and if last 6 character (seq[-6:]) is "<body>" (line 8), we begin the html code tracking.

For CSS code tracking, we track selector-property tuples since `<selector,property>` pairs uniquely define element styling specifications. Repeated `<selector,property>` represent redundant declarations, making them ideal candidates for repetition penalty. We also perform duplicate token detection on individual selector, property, and value strings, because there may be duplication of text in selector, property and value string. Algorithm 2 illustrates this process: Lines 3-17

Algorithm 1 Repetition Frequency Tracking

Initialize: $css\_counts \leftarrow \{\}$; $html\_counts \leftarrow \{\}$; $text\_counts \leftarrow \{\}$
$parser\_state \leftarrow INITIAL$; $\lambda \leftarrow$ exponential decay parameter
while MLLM generating next token **do**
    token ← get_next_token()
    $sequence += token$
    if $sequence[-7:]$ = "<style>" **then**
        $parser\_state \leftarrow CSS$
    else if $sequence[-6:]$ = "<body>" **then**
        $parser\_state \leftarrow HTML$
    end if
    if $parser\_state = CSS$ **then**
        CSSParser($token$, css_counts, $text\_counts$)
    else if $parser\_state = HTML$ **then**
        HTMLParser($token$, $html\_counts$, $text\_counts$)
    end if
    Exponential_Penalty_Strategy($token$, css_counts, $html\_counts$, $text\_counts$, λ)
end while

Algorithm 2 CSS Parser

Function CSS_PARSER($token$, css_counts, $text\_counts$)
static $cur\_selector \leftarrow$""; $cur\_property \leftarrow$""; $cur\_value \leftarrow$""; $parse\_state \leftarrow SELECTOR$
if $token$ = "{" **then**
    $css\_counts[selector][cur\_selector] += 1$
    $parse\_state \leftarrow PROPERTY$
else if $token$ = ":" **then**
    $css\_counts[property][cur\_property] += 1$
    $css\_counts[selector\_property][(cur\_selector, cur\_property)] += 1$
    $parse\_state \leftarrow VALUE$
else if $token$ = ";" OR $token$ = "}" **then**
    $css\_counts[value][cur\_value] += 1$
    if $token$ = "}" **then**
        $parse\_state \leftarrow SELECTOR$
    else
        $parse\_state \leftarrow PROPERTY$
    end if
else
    if $parse\_state = SELECTOR$ **then**
        $cur\_selector += token$
        $String\_Repeat\_Detection(cur\_selector, text\_counts)$
    else if $parse\_state = PROPERTY$ **then**
        $cur\_property += token$
        $String\_Repeat\_Detection(cur\_property, text\_counts)$
    else if $parse\_state = VALUE$ **then**
        $cur\_value += token$
        $String\_Repeat\_Detection(cur\_value, text\_counts)$
    end if
end if

demonstrate the token-by-token parsing logic where each input token undergoes type classification to determine the appropriate parsing context. Upon encountering an opening brace ({), the parser increments the frequency counter for the current selector and transitions to the `PROPERTY` state. A colon token (`:`) triggers simultaneous incrementation of the property count and the combined `<selector,property>` tuple count, followed by a state transition to `VALUE`. The parser handles

Algorithm 3 HTML Parser

```
Function HTML_PARSER(token, html_counts, text_counts)
static cur_tag_property_value ←""; cur_content ←"";
static parse_state ←""
if token = "<" then
    parse_state ← TAG_PROPERTY_VALUE
    if cur_tag_property_value != "" then
        html_counts[quadruple][(cur_tag_property_value, cur_content)]+ = 1
    end if
else if token = ">" then
    parse_state ← CONTENT
else
    if parse_state = TAG_PROPERTY_VALUE then
        cur_tag_property_value ← cur_tag_property_value + token
    else if parse_state = CONTENT then
        cur_content ← cur_content + token
        String_Repeat_Detection(token, cur_content, text_counts)
    end if
end if
```

statement termination through semicolon (;) and closing brace (}) tokens, which increment the current value's frequencies and initiate transitions to either `SELECTOR` or `PROPERTY` states, respectively. For HTML code tracking, we track the quadruples `<tag,property,value,content>`, because we observe that this structure is repeated in the code. As shown in Algorithm 3, the parser maintains static variables for current tag-property-value combinations (*cur_tag_property_value*) and content accumulation (*cur_content*), along with a parsing state indicator (*parse_state*). Upon encountering opening angle brackets ("<"), the parser transitions to `TAG_PROPERTY_VALUE` state and conditionally updates the frequency count for HTML quadruples if a valid tag-property-value combination exists. Closing angle brackets (">") trigger transitions to `CONTENT` state, enabling content parsing within HTML elements. The *String_Repeat_Detection* function apply the repeated substring detection algorithm [38] to record the number of times repeated substrings appear.

4.3.2 Exponential Penalty Strategy. The decoding process for LLM begins with the computation of logits of the token. Given the hidden state $\mathbf{h}_t \in \mathbb{R}^d$ at position t, the model computes unnormalized scores (logits) for each token in the vocabulary through a linear transformation: $\mathbf{z}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$, where $\mathbf{z}_t \in \mathbb{R}^{|V|}$ represents the logits for all tokens in vocabulary V, $\mathbf{W} \in \mathbb{R}^{|V|\times d}$ is the output projection matrix, and $\mathbf{b} \in \mathbb{R}^{|V|}$ is the bias vector. These logits are subsequently transformed into a probability distribution over the vocabulary using the softmax function: $P(w_t = v_i|\mathbf{x}_{<t}) = \frac{\exp(z_{t,i})}{\sum_{j=1}^{|V|}\exp(z_{t,j})}$, where w_t denotes the token at position t, v_i represents the i-th token in vocabulary V, $z_{t,i}$ is the corresponding logit, and $\mathbf{x}_{<t}$ encompasses all preceding tokens in the sequence. Finally, LLM will select the token with the highest probability to output. The exponential penalty strategy applies a decay mechanism that becomes more aggressive as repetition frequency increases. When duplicates occur, we will impose the following penalties on the subsequent s tokens' logits to suppress repeated token:

$$\tilde{z}_i = z_i \cdot \lambda^c, \quad \forall i \in \{1, 2, \ldots, s\}, \tag{5}$$

where c is the number of repetitions, $\lambda < 1$ is the decay factor. We modify the MLLM decoding by monitoring token frequencies. Once repetitions exceed a suppression step threshold s, we penalize

the logits of the repeated token by multiplying them by the decay factor λ before probability conversion, and maintain this penalty for the subsequent s steps.

5 Experiment Setup

5.1 Models

Since we need to compress the input token after the visual encoder and adjust the output decoding process, we select the popular open-source MLLMs llava-v1.6 (7B, 34B) [18] for the experiment. To improve the reproducibility of experimental results, we set the temperature of all models to 0 and fixed the random seeds to 2026. All the experiments are conducted on a Linux server equipped with 4 NVIDIA A800 80GB GPUs. We set the maximum output token length to 4096 and use the direct prompt in Design2Code [25] for evaluation.

5.2 Evaluation Datasets

We use two widely used UI2Code datasets: (1) **Design2Code [25]**: Design2Code is a pioneering benchmark dataset comprising 484 real-world webpage screenshots paired with their corresponding front-end code (HTML/CSS), created to rigorously evaluate multimodal large language models' ability to generate visually accurate implementations of design layouts. (2) **WebCode2M [9]**: WebCode2M is a high-quality, large-scale and real-word datasets for training and evaluating the automated webpage code generation task. We randomly select 100 webpages from WebCode2M-short, WebCode2M-mid and WebCode2M-long as the testset.

5.3 Evaluation Metrics

5.3.1 Performance Metrics. **High-level Similarity.** High-level metrics assess the overall fidelity of webpage appearance and code structure. (1) **CLIP Score** [23]: visual similarity between the reference image (I_R) and the generated image (I_G) is measured by CLIP embedding similarity, denoted as $\textbf{CLIP}(I_R, I_G)$. Features are extracted using CLIP-ViT-B/32. (2) **BLEU** [22]: code similarity is evaluated by calculating the precision of n-grams in the generated HTML code with respect to the reference code, weighted by a brevity penalty to account for length differences. Here, we don't apply CodeBERT [7] with cosine similarity to calculate the code similarity, because: (1) it is trained on six programming languages (i.e., Go, Java, JavaScript, PHP, Python, and Ruby) and does not include HTML/CSS; and (2) its maximum input length is limited to 512 tokens, which is insufficient for UI code, as generated webpages typically require thousands of tokens.

Low-level Similarity. High-level measures alone are insufficient to capture fine-grained discrepancies. To provide a more detailed evaluation, we adopt element-matching metrics introduced by Si et al. [25], which assess similarity across text, position, and color. Given reference and generated webpage screenshots, visual element blocks are detected and aligned using the Jonker-Volgenant assignment algorithm. The quality of the matches is then quantified by: (1) **Block-match**: the ratio of matched block areas to total block areas, penalizing missing or hallucinated elements; (2) **Text similarity**: character overlap measured by the Sørensen-Dice coefficient; (3) **Color similarity**: perceptual differences computed using the CIEDE2000 formula; (4) **Position similarity**: alignment accuracy based on block center locations.

5.3.2 Efficiency Metrics. We evaluate the efficiency of MLLMs on the UI2Code task using the following metrics, which capture both computational complexity and temporal performance: (1) **Compression Ratio**: $R = \frac{C_{compressed}}{C_{original}}$, where $C_{compressed}$ and $C_{original}$ denote the number of visual tokens in the compressed and original UI image respectively. For our method, since the proportion of tokens compressed on each UI image is different, we report the average compression rate of the

entire dataset. (2) **Floating Point Operations (FLOPs):** this metric quantifies the total number of arithmetic operations performed by the model during the complete inference process. Since compression is applied subsequent to the visual encoder, we specifically calculate the FLOPs of the LLM backbone to ensure fair comparison across different architectures. (3) **Prefill Time:** This measures the latency required to generate the first output token, encompassing input preprocessing, key-value (KV) cache initialization, and the initial forward pass computation. (4) **Inference Time:** This represents the end-to-end latency from input reception to complete output sequence generation, providing a comprehensive measure of the model's real-world deployment efficiency.

5.3.3 Human Evaluation Metrics. While automatic metrics provide a fine-grained assessment of model performance, evaluating user perceptions of the generated webpages is also essential. Following Design2Code [25], we conduct a human evaluation, as detailed below. (1) **Participant Information and Recruitment**. We recruit five annotators through a local university mailing list. All annotators hold at least a B.S. degree in computer science or software engineering and have more than two years of web-development experience. (2) **Evaluation Procedure**. Step 1. We randomly sample 50 pages from Design2Code and 50 from WebCode2M. Each annotator evaluate 100 unique pairs (Vanilla vs. one compression method) in a single round. Step 2. Annotators are required to carefully read the pairwise comparison guidelines[1]. Step 3. For each pair, annotators select "Example 1 better", "Example 2 better", or "Tie", following the detailed guidelines. Annotators are blind to which method produced each webpage, and pairs are presented in randomized order. (3) **Inter-rater Reliability**. To assess the consistency and reliability of the human annotations, we compute Krippendorff's alpha. The resulting value is 0.8308, which exceeds the commonly adopted threshold of 0.8, indicating a high level of inter-annotator agreement.

5.4 Baselines

- **Vanilla** denotes the original model without any token compression.
- **Random** means randomly selecting a certain proportion of tokens from the UI image.
- **FastV** [4] prunes low-attention visual tokens after a designated LLM layer.
- **Pdrop** [37] partition the MLLMs into several stages and drop part of the image tokens at the end of each stage with a pre-defined ratio. The dropping is based on the token similarity calculation.
- **VisionZip** [40] selects dominant tokens by visual encoder's attention score, discarding a certain proportion of visual tokens before inputting the MLLMs.

5.5 Research Questions

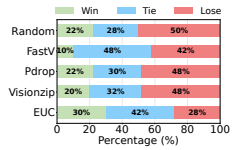
- (RQ1) **Performance**. Can EfficientUICoder effectively compress token while preserving the UI2Code performance?
- (RQ2) **Efficiency**. What is the efficiency benefit of EfficientUICoder in UI2Code tasks?
- (RQ3) **Ablation study**. How does different parts contribute to EfficientUICoder's performance?
- (RQ4) **Parameter study**. How do key parameters affect the performance of EfficientUICoder?
- (RQ5) **Case Study.** Why does EfficientUICoder work?

6 Experiment Results

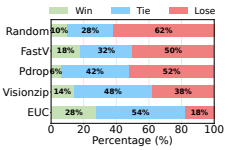
6.1 Performance Comparison (RQ1)

Table 2 presents the performance comparison of different compression methods. For EfficientUICoder, after applying the ELTC and RTR modules, we achieve compression ratios of 60.36% on the Design2Code dataset and 55.86% on the Webcode2M dataset. To ensure fair comparison, we

[1]https://github.com/WebPAI/EfficientUICoder/blob/main/assets/Human_Evaluation.md

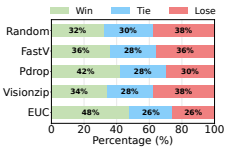


(a) Llava-v1.6-7b.

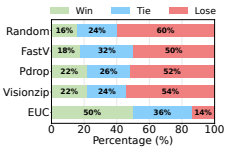


(b) Llava-v1.6-34b.

Fig. 6. Human Evaluation on Design2Code.



(a) Llava-v1.6-7b.



(b) Llava-v1.6-34b.

Fig. 7. Human Evaluation on WebCode2M.

configure all baseline methods to achieve the same compression ratios. We can make the following observations: (1) **EfficientUICoder demonstrates superior performance while compressing 55%-60% of tokens.** Our method consistently outperforms existing compression techniques across most evaluation metrics. This superiority stems from our structured approach to token selection: while baseline methods either select tokens randomly or rely solely on attention mechanisms without considering UI structure, EfficientUICoder's ELTC module performs selection based on UI elements and layout hierarchy, while the RTR module incorporates semantic importance. Remarkably, our method even surpasses the vanilla version in several metrics. We attribute this improvement to two factors: first, EfficientUICoder's token selection mechanism enables the model to focus more effectively on critical UI elements and layout information; second, the ADTS component efficiently reduces duplicate tokens, facilitating the generation of more coherent and valid webpages. (2) **EfficientUICoder significantly reduces redundant code generation.** The RS metric demonstrates substantial improvements: for the 7b model, redundant samples decrease from 148 to 64 on Design2Code (56.8% reduction) and from 39 to 15 on Webcode2M (61.5% reduction). These results validate that the ADTS module effectively suppresses redundant code generation, leading to more concise and efficient outputs. (3) EfficientUICoder consistently outperforms baselines across datasets and model sizes. Compared with vanilla, performance differences are insignificant when vanilla averages are higher, while several gains are statistically significant when EfficientUICoder leads, indicating that it maintains or even surpasses near-uncompressed performance.

Human Evaluation. We conduct human evaluation on 50 webpages from Design2Code and 50 webpages from WebCode2M. Fig. 6 and Fig. 7 (EUC denotes EfficientUICoder) present the pairwise preference evaluation results, where five annotators compare different methods against the Vanilla using majority voting. Higher win rates and lower loss rates indicate superior code quality as assessed by human evaluators. The results demonstrate that EfficientUICoder consistently outperforms all baseline methods across both datasets. Importantly, these human evaluation results corroborate our automatic metrics, confirming the validity of our automated metrics. As shown in Table 3, the consistently low p-values indicate that human evaluators judge EfficientUICoder's outputs to be significantly higher quality than those of other compression methods.

Answer to RQ1: EfficientUICoder effectively achieves 55%-60% image token compression and 56%-94% code redundancy reduction while maintaining or improving generation performance.

6.2 Efficiency Comparison (RQ2)

Table 4 presents the efficiency analysis of different compression methods on Llava-v1.6-34b across both datasets. Several key observations emerge from this analysis.(1) **EfficientUICoder achieves superior computational savings.** Our method demonstrates the highest FLOPs reduction of 44.9% on Design2Code and 42.8% on Webcode2M, significantly outperforming other techniques.

Table 2. Performance Comparison of different Compression Methods on Two Datasets. On the Design2Code dataset, the compression ratio is 60.36%. On the Webcode2M dataset the compression ratio is 55.86%. Bold values indicate the optimal performance, and underlined values indicate the second best performance. RS is the number of samples with redundant codes. Rows marked with p-value show statistical significance of Mann-Whitney U test comparing EfficientUICoder vs. each baseline ($p < 0.05$ indicates statistical significance). EUC denotes EfficientUICoder.

Method	Design2Code							Webcode2M						
	Block	Text	Position	Color	CLIP	BLEU	RS	Block	Text	Position	Color	CLIP	BLEU	RS
Llava-v1.6-7b														
Vanilla	**0.3675**	0.7275	0.5655	0.5124	0.6916	0.1667	148	**0.2843**	0.7408	0.5728	**0.5056**	0.7123	0.1190	39
p-value	2.6e-1	5.5e-2	4.7e-2	8.6e-1	2.3e-2	2.0e-2	-	7.4e-1	7.3e-1	2.1e-1	7.3e-1	1.1e-1	1.4e-1	-
Random	0.1699	0.5998	0.5001	0.4599	0.6492	0.0413	144	0.1449	0.5854	0.4829	0.4418	0.6373	0.0301	34
p-value	2.2e-17	7.4e-22	8.3e-10	7.4e-3	8.1e-4	1.2e-32	-	1.6e-4	7.9e-6	1.4e-4	2.5e-1	1.5e-2	2.5e-7	-
FastV	0.2972	0.6141	0.4769	0.4209	0.5886	0.1154	132	0.2052	0.6785	0.5291	0.4546	0.6594	0.0832	40
p-value	1.2e-3	6.3e-3	7.0e-9	8.0e-6	2.2e-8	2.1e-12	-	5.6e-2	4.1e-1	1.8e-2	3.0e-1	2.5e-2	9.1e-3	-
Pdrop	0.1907	0.6342	0.5108	0.4657	0.6587	0.0602	141	0.1657	0.6738	0.5393	0.4628	0.6866	0.0698	35
p-value	5.5e-14	1.5e-13	2.0e-9	1.5e-2	2.6e-4	2.6e-21	-	4.4e-3	2.2e-2	6.9e-3	3.5e-1	1.0e-1	1.1e-2	-
Visionzip	0.3242	0.6933	0.5357	0.4877	0.6694	0.1388	204	0.1973	0.6923	0.5607	0.4625	0.6799	0.0705	53
p-value	3.0e-1	6.9e-1	5.5e-5	3.7e-1	3.5e-3	1.4e-6	-	5.6e-2	3.5e-1	1.4e-1	6.1e-1	1.2e-2	7.6e-4	-
EUC	0.3374	**0.7333**	**0.5939**	**0.5125**	**0.7172**	**0.1855**	**64**	0.2718	**0.7445**	**0.6170**	0.4909	**0.7393**	**0.1338**	**15**
Llava-v1.6-34b														
Vanilla	**0.5516**	0.7914	0.6292	0.5842	0.7343	**0.2914**	136	**0.4688**	**0.8207**	**0.6441**	**0.5546**	**0.7554**	**0.2689**	33
p-value	9.2e-2	1.1e-3	9.1e-1	5.7e-1	8.6e-2	1.1e-1	-	5.4e-1	1.9e-1	1.2e-1	8.9e-1	5.1e-1	9.1e-1	-
Random	0.2108	0.5406	0.4629	0.4297	0.5845	0.0552	98	0.1291	0.4769	0.4491	0.3385	0.5607	0.0443	30
p-value	1.5e-50	1.5e-53	2.8e-30	6.7e-24	8.8e-23	6.2e-57	-	1.5e-12	2.5e-15	6.9e-6	2.5e-8	3.0e-8	7.5e-14	-
FastV	0.3161	0.6913	0.5789	0.5289	0.7085	0.1025	125	0.2719	0.6691	0.5518	0.4620	0.6878	0.0774	34
p-value	2.8e-22	4.1e-24	1.1e-12	4.3e-9	3.5e-13	2.4e-32	-	5.3e-5	1.5e-8	1.9e-3	3.3e-4	2.7e-5	1.0e-8	-
Pdrop	0.4117	0.7071	0.5744	0.5124	0.6876	0.1577	111	0.3747	0.6968	0.5750	0.4644	0.6892	0.1599	23
p-value	1.0e-29	1.7e-28	9.3e-27	4.3e-27	1.0e-29	3.0e-35	-	3.1e-4	1.7e-6	4.2e-4	4.6e-6	7.1e-6	7.9e-6	-
Visionzip	0.4824	0.8040	0.6333	0.6013	0.7620	0.2313	75	0.4176	0.7077	0.5959	0.4886	0.6903	0.2117	22
p-value	5.2e-8	6.4e-5	9.6e-13	2.5e-8	1.4e-15	1.6e-9	-	4.9e-4	5.7e-6	9.1e-4	1.9e-5	2.2e-6	5.3e-5	-
EUC	0.5318	**0.8113**	**0.6643**	**0.6110**	**0.7864**	0.2547	**16**	0.4382	0.7533	0.5907	0.5313	0.7270	0.2634	**2**

Table 3. Human evaluation's Mann-Whitney-U test results between EUC and baselines. EUC denotes EfficientUICoder, "Y" denotes significant (p-value<0.05) and "N" denotes not significant (p-value>=0.05).

Dataset	EUC vs. random	EUC vs. fastv	EUC vs. pdrop	EUC vs. visionzip
Design2Code	1.88e-05 (Y)	3.22e-05 (Y)	8.27e-05 (Y)	5.60e-05 (Y)
WebCode2M	1.63e-05 (Y)	3.93e-03 (Y)	3.82e-05 (Y)	1.49e-02 (Y)

This translates to substantial reductions in computational requirements and energy consumption. (2) **EfficientUICoder achieves significant reduction in output tokens.** EfficientUICoder dramatically reduces generated tokens while maintaining quality: from 2041 to 1195 tokens on Design2Code (41.4% reduction) and from 2116 to 1340 on Webcode2M (36.7% reduction). This indicates effective elimination of redundant code patterns. (3) **Superior inference and prefill acceleration.** Our method achieves the most significant inference time improvements: 48.8% reduction on Design2Code and 45.1% on Webcode2M, substantially exceeding other methods. Additionally, EfficientUICoder demonstrates competitive prefill time performance with 46.6% and 45.9% improvements respectively. This dual acceleration in both prefill and inference phases is crucial for real-world deployment scenarios. EfficientUICoder also shows statistically significant

Table 4. Efficiency comparison on *Llava-v1.6-34b* for two datasets. GT denotes the number of generated tokens, PT denotes the prefill time, and IT denotes the inference time. P-value of Mann-Whitney-U test between EfficientUICoder and baselines are reported in the row below each baselines. EUC denotes EfficientUICoder, "Y" denotes significant (p-value<0.05) and "N" denotes not significant (p-value>=0.05).

Method	Design2Code				Webcode2M			
	FLOPs (T)	GT	PT (ms)	IT (s)	FLOPs (T)	GT	PT (ms)	IT (s)
Vanilla	184.1	2041	1012	160	195.0	2116	1102	173
p-value	5.53e-129 (Y)	1.10e-14 (Y)	1.18e-158 (Y)	3.58e-28 (Y)	1.53e-27 (Y)	1.42e-03 (Y)	2.89e-34 (Y)	5.34e-06 (Y)
Random	121.7 ↓ 33.9%	1710 ↓ 16.2%	549 ↓ 45.8%	122 ↓ 23.8%	144.3 ↓ 26.0%	2144 ↑ 1.3%	607 ↓ 44.9%	156 ↓ 9.8%
p-value	1.48e-04 (Y)	9.15e-09 (Y)	5.79e-06 (Y)	3.41e-11 (Y)	5.58e-02 (N)	1.17e-02 (Y)	2.09e-02 (Y)	2.64e-03 (Y)
FastV	119.9 ↓ 34.9%	1864 ↓ 8.7%	606 ↓ 40.1%	130 ↓ 18.8%	138.0 ↓ 29.2%	2136 ↑ 0.9%	734 ↓ 33.4%	158 ↓ 8.7%
p-value	9.43e-07 (Y)	1.46e-02 (Y)	6.93e-25 (Y)	4.38e-03 (Y)	1.33e-02 (Y)	7.31e-01 (N)	3.85e-14 (Y)	5.78e-01 (N)
Pdrop	122.7 ↓ 33.4%	1958 ↓ 4.1%	**491** ↓ 51.5%	137 ↓ 14.4%	131.2 ↓ 32.7%	1980 ↓ 6.4%	**582** ↓ 47.2%	144 ↓ 16.8%
p-value	7.72e-18 (Y)	7.62e-08 (Y)	1.21e-33 (Y)	2.71e-11 (Y)	3.21e-04 (Y)	8.30e-03 (Y)	9.16e-03 (Y)	1.68e-03 (Y)
Visionzip	125.4 ↓ 31.9%	1807 ↓ 11.5%	544 ↓ 46.2%	127 ↓ 20.6%	137.2 ↓ 29.6%	1960 ↓ 7.4%	592 ↓ 46.3%	142 ↓ 17.9%
p-value	2.71e-03 (Y)	3.76e-05 (Y)	6.53e-06 (Y)	1.07e-07 (Y)	1.24e-02 (Y)	1.02e-02 (Y)	9.56e-03 (Y)	1.88e-03 (Y)
EUC	**101.5** ↓ 44.9%	**1195** ↓ 41.4%	540 ↓ 46.6%	**82** ↓ 48.8%	**111.6** ↓ 42.8%	**1340** ↓ 36.7%	596 ↓ 45.9%	**95** ↓ 45.1%

superiority across both datasets when compared to compression-based baselines and vanilla. It denotes that EfficientUICoder demonstrates significant efficiency improvements.

Answer to RQ2: EfficientUICoder achieves superior efficiency improvements across all metrics, reducing computational cost by up to 44.9%, generated tokens by up to 41.4%, prefill time by up to 46.6%, and inference time by up to 48.8%.

6.3 Ablation Study (RQ3)

EfficientUICoder comprises three core components: Element and Layout-aware Token Compression (**ELTC**), Region-aware Token Refinement (**RTR**), and Adaptive Duplicate Token Suppression (**ADTS**). To evaluate the contribution of each component, we conduct an ablation study using five variants of EfficientUICoder(C_0-C_5). Y indicates the inclusion of the corresponding component, while X denotes its removal. Configuration C_5 represents the complete EfficientUICoder with all three components enabled. Since RTR requires the token selection from ELTC to function properly, we replace RTR with a high-attention-score token selection strategy when ELTC is disabled to ensure fair comparison. First, Table 5 demonstrates that each individual component contributes positively to the overall performance of EfficientUICoder. The combination of all three components (C_4) achieves superior performance and efficiency compared to any partial configuration, highlighting the complementary nature of these components in optimizing UI code generation tasks. Second, C_1 and C_4 provide ablations that separately examine input compression (ELTC + RTR) and the repetition-suppression mechanism (ADTS). The results show: (1) Removing input compression substantially increases FLOPs, prefill time, and overall inference latency, while also degrading UI2Code performance. Redundant visual tokens distract the model from key UI elements, reducing accuracy, and the inflated number of image tokens further increases computational cost (FLOPs), leading to slower prefill and inference. (2) Removing ADTS decreases the performance and increases inference time, because duplicate contents affect the webpages' appearance and endless repetition severely hinders UI2Code's end-to-end inference time latency. In summary, input compression and repetition suppression work in tandem: compression streamlines visual inputs, while ADTS ensures clean, non-repetitive outputs. Removing either component degrades both efficiency and accuracy, showing that the two mechanisms are complementary.

Table 5. The performance and efficiency of 5 variants (C_0-C_5) on *Llava-v1.6-7b* for WebCode2M.

Datasets	ELTC	RTR	ADTS		Block	Text	Position	Color	CLIP	BLEU	FLOPs (T)	PT (ms)	IT (s)
WebCode2M	X	X	X	C_0	0.2843	0.7408	0.5728	**0.5056**	0.7123	0.1190	29.3	242	37
	X	X	Y	C_1	**0.2943**	0.7431	0.5685	0.5018	0.7191	0.1256	28.1	259	34
	X	Y	Y	C_2	0.2247	0.7114	0.5924	0.4716	0.7208	0.1187	**20.1**	142	33
	Y	X	Y	C_3	0.2277	0.6839	0.5685	0.4813	0.6690	0.1262	20.6	138	34
	Y	Y	X	C_4	0.2454	0.7002	0.5766	0.4772	0.6862	0.0890	23.6	**136**	45
	Y	Y	Y	C_5	0.2718	**0.7445**	**0.6172**	0.4909	**0.7393**	**0.1338**	20.4	141	**32**

Answer to RQ3: Each component of EfficientUICoder contributes positively to performance.Input compression streamlines visual inputs, while ADTS suppresses repetition; removing either component degrades both efficiency and accuracy, indicating their complementarity. Consequently, the full model achieves the best results, outperforming any subset of components.

Table 6. Performance Comparison on WebCode2M. The compression ratio is 63.42%. Bold values indicate the optimal performance, and underlined values indicate the second best performance. RS is the number of samples with redundant codes.

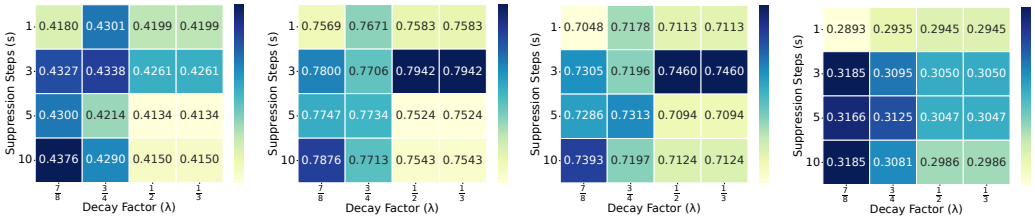
Method	Webcode2M									
	Block	Text	Position	Color	CLIP	BLEU	RS	FLOPs(T)	PT(ms)	IT(s)
Qwen2.5-VL-32B										
Vanilla	**0.8683**	0.9621	0.7996	0.7544	0.8624	0.6950	8	84.5	976	92
Random	0.6653	0.9001	0.8158	**0.7987**	0.7967	0.4086	3	35.4	510	80
Visionzip	0.6283	0.9054	0.8155	0.7718	0.8715	0.4849	5	34.6	585	77
EfficientUICoder	0.8661	**0.9692**	**0.8410**	0.7932	**0.8802**	**0.6999**	**1**	**34.5**	**492**	**74**

6.4 Parameter Study (RQ4)

We sample 100 webpages in Design2Code and 50 webpages in WebCode2M for parameter study.

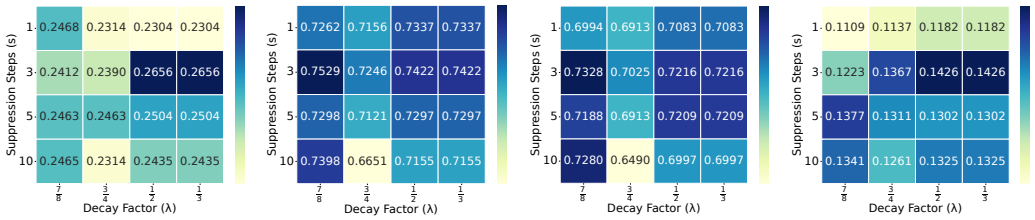
6.4.1 Other Backbones. We additionally evaluate EfficientUICoder using Qwen2.5-VL Series [2], another state-of-the-art open-source MLLM. Since some baselines (fasv and pdrop) lack Qwen-based implementations, we compare our model against vanilla, random and visionzip. We report Qwen2.5-VL-32B's performance on WebCode2M in Table 6, indicating that **EfficientUICoder achieves the best overall webpage quality and efficiency on Qwen2.5-VL-32B, further demonstrating EfficientUICoder's generalizability.**

6.4.2 The Suppression Step s and Decay Factor λ. We conduct a comprehensive parameter study by setting $s \in [1, 3, 5, 10]$ and $\lambda \in [\frac{7}{8}, \frac{3}{4}, \frac{1}{2}, \frac{1}{3}]$. As shown in Fig. 9 and Fig. 8, EfficientUICoder achieves optimal performance when $s = 3$ and $\lambda = \frac{1}{2}$. A suppression step that is too small ($s < 3$) fails to sufficiently penalize consecutive duplicate tokens, while an excessively large step ($s > 3$) results in over-penalization. Similarly, when the decay factor λ is too large (close to 1), the logits of redundant code are not effectively reduced, whereas smaller values ($\lambda \leq \frac{1}{2}$) achieve similar effects in encouraging non-redundant token selection. Fig. 10 demonstrates that when $s \geq 3$, inference time is significantly reduced to approximately 30 seconds. Our analysis reveals that larger decay factors should be paired with larger penalty steps, while smaller decay factors work better with smaller penalty steps, achieving an optimal balance between penalty intensity and frequency to avoid under- or over-penalization of redundant content.



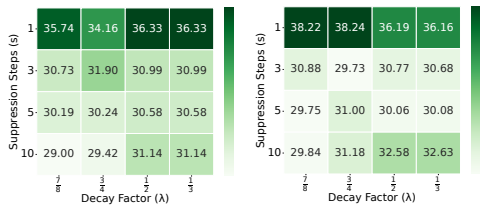
(a) Block Match. (b) Text Match. (c) CLIP Score. (d) Bleu.

Fig. 8. Suppression step s and decay factor λ analysis on Design2Code dataset.



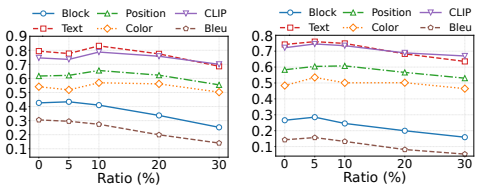
(a) Block Match. (b) Text Match. (c) CLIP Score. (d) Bleu.

Fig. 9. Suppression step s and decay factor λ analysis on WebCode2M dataset.



(a) Design2Code. (b) WebCode2M.

Fig. 10. Inference time under different λ and r.

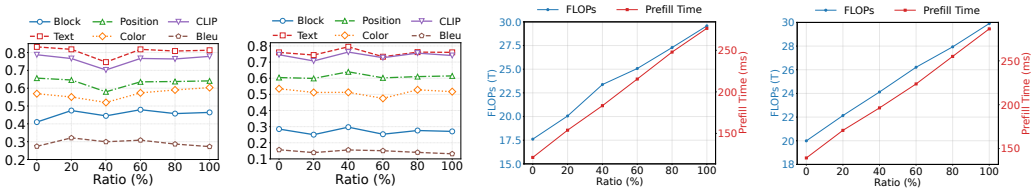


(a) Design2Code. (b) WebCode2M.

Fig. 11. Refinement ratio r analysis.

6.4.3 The Refinement Token Ratio r. Maintaining the optimal parameter combination ($s = 3, \lambda = \frac{1}{2}$), we investigate the influence of the refinement ratio by setting $r \in [5\%, 10\%, 20\%, 30\%]$. Fig. 11 shows that EfficientUICoder achieves the best performance on Design2Code and WebCode2M datasets when $r = 10\%$ and $r = 5\%$, respectively. The performance exhibits an inverted-U shape as r increases: initially improving as EfficientUICoder effectively discards redundant tokens in the ELTC region while incorporating important tokens from the non-ELTC region. However, excessively large values of r lead to performance degradation due to the inadvertent removal of key tokens within the ELTC region, which represents crucial UI elements and layout information.

6.4.4 The Proportion of Increased Tokens a. To further validate the effectiveness of our token selection strategy, we explore the impact of adding a certain proportion of tokens from the initially unselected set. We divide the unselected set into five parts, and add $\frac{1}{5}$ of the tokens each time ($a \in [20\%, 40\%, 60\%, 80\%, 100\%]$). Fig. 12 reveals that increasing a does not yield significant performance improvements but instead causes fluctuations. This observation confirms the effectiveness of



(a) Performance on D2C. (b) Performance on WC2M. (c) Efficiency on D2C. (d) Efficiency on WC2M.

Fig. 12. The performance and efficiency on two datasets when adding unselected token. D2C denotes Design2Code and WC2M denotes WebCode2M.

EfficientUICoder's token selection strategy, demonstrating that adding more tokens provides only marginal performance gains while substantially increasing computational costs.

Answer to RQ4: A moderate suppression step $s = 3$ and decay factor $\lambda = \frac{1}{2}$ achieve a good balance between under- and over-penalization. The refinement ratio r works best at small values (5-10%), effectively removing redundancy without discarding key UI elements. EfficientUICoder shows generalizability across different MLLM backbones.

6.5 Case Study (RQ5)

We present two cases generated by Llava-v1.6-34b from the Design2Code dataset with visualized token selection results to demonstrate the effectiveness of EfficientUICoder. (1) EfficientUICoder effectively selects tokens that encompass critical UI elements and structural components. As shown in Fig.13, the webpage generated by EfficientUICoderachieves quality comparable to the Vanilla approach while significantly outperforming VisionZip. Detailed analysis reveals that VisionZip fails to generate essential elements including contact information (name, phone number, email), interactive components (message input boxes), and key section headers such as "Our Services" and "Why Choose Us". Fig.14 illustrates the token selection comparison: EfficientUICoder's selections cover nearly all UI elements, whereas VisionZip not only omits crucial text and component tokens but also includes numerous redundant background tokens. (2) EfficientUICoder effectively prevents the generation of duplicate code structures. Fig. 15 demonstrates a scenario where the Vanilla method becomes trapped in continuously generating the paragraph element "Sure enough,...", resulting in excessive webpage length and preventing the generation of subsequent elements. In contrast, EfficientUICoder successfully avoids this repetitive generation through its ADTS module, ensuring balanced and complete code synthesis.

Answer to RQ5: EfficientUICoder works through two key mechanisms: (1) Effective token selection that covers critical UI elements while filtering redundant tokens, and (2) ADTS's redundancy prevention that avoids repetitive code generation, ensuring valid webpage.

7 Threats to Validity

(1) *Selection of backbone models.* We adopt widely used MLLMs, LLaVA and Qwen, with 7B and 30B parameters as backbones. Commercial LLMs are excluded because their closed-source nature prevents modifications to the encoding and decoding processes. (2) *Metric bias.* We use BLEU and CLIP scores to measure code similarity and visual consistency, along with fine-grained metrics.

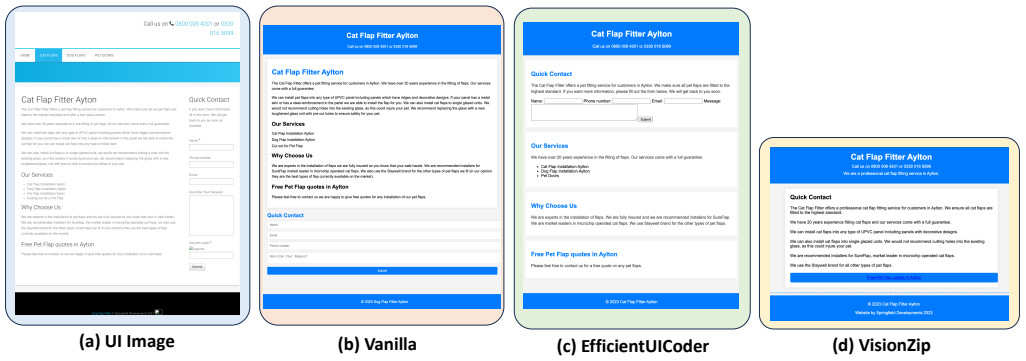


Fig. 13. Case study of webpages generated by Vanilla, EfficientUICoder and VisionZip.

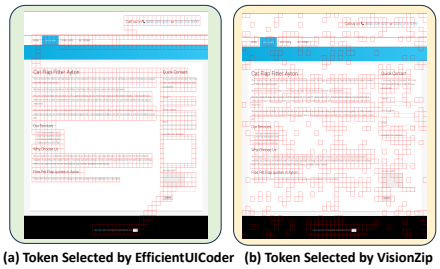


Fig. 14. Token selection results.

Fig. 15. Output redundancy case study.

Since no standardized UI2Code evaluation framework exists, these metrics may not fully capture task nuances; thus, we supplement them with human evaluation (Section 6.1).

8 Related Work

UI Code Generation Method. Early UI code generation methods relied on deep learning and computer vision techniques, such as CNN-based encoder-decoder frameworks [1, 3, 5] and OCR/object-detection-based systems [14, 19]. Recent studies have shifted toward MLLM-based approaches to improve layout fidelity and element completeness. DCGen [30] adopts a divide-and-conquer strategy, DeclarUI [44] integrates segmentation with transition graphs, and UICopilot [11], LayoutCoder [32] and LatCoder [10] further enhance generation through hierarchical and layout-aware modeling. TDDev [29] introduces a test-driven development to enable the generation of full-stack web applications. ComUICoder [33] applies semantic-aware segmentation and merging techniques for component-based UI code generation, improving code reusability. However, these methods mainly focus on generation quality, while overlooking the computational overhead challenge.

UI Code Generation Benchmark. Early works like WebSight and Web2Code [41] pioneered HTML code synthesis and systematic assessment through the Webpage Code Generation Benchmark (WCGB), though both relied on synthetic data. Design2Code [25] introduced the first real-world benchmark with 484 manually curated web pages from Common Crawl, while WebCode2M [9] scaled this to 20,000 samples for comprehensive training and evaluation. Specialized benchmarks

have emerged targeting specific aspects: Interaction2Code [34] for interactive generation, MRWeb [28] for multi-page resource-aware websites, and DesignBench [35] for multi-task framework-based UI generation, editing, and repair. These benchmarks collectively provide comprehensive evaluation frameworks for different dimensions of MLLM web development capabilities.

Token Compression for MLLMs. Existing token compression methods mainly reduce redundant visual tokens based on attention scores or token similarity. For example, FastV [4] drops half of visual tokens at the second layer during inference, Pdrop [37] performs stage-wise token pruning with predefined ratios, SparseVLM [43] leverages text-guided attention for adaptive pruning, and VisionZip [40] filters tokens using visual encoder attention scores. However, these methods are not well suited for UI2Code tasks, as they ignore UI element semantics and do not address redundancy in generated outputs.

Repetitive Content Compression. Repetition penalty [13, 31] is a widely adopted technique for mitigating content repetition during decoding by penalizing previously generated tokens. ARP [8] proposes a rebalanced encoding strategy to alleviate repetitive content during the training phase. Repetition dropout [16] reduces repetitive content by selectively dropping attention to repetitive tokens in the training data. ADTS differs from above methods in four ways: (1) **Structure-aware counters**: instead of plain token strings tracking, we track <selector, property> tuples for CSS, 4-tuples <tag, attr, value, text> for HTML and substring for text content. (2) **Dynamic scope**: penalty is applied only inside the corresponding syntactic scope (style block or body), avoiding over-suppression of other contents. (3) **Exponential decay**: ADTS also incorporates an exponential penalty parameter, the more repetitions, the more pronounced the penalty, instead of completely suppressing it whenever repetition occurs. (4) **Lightweight**: ADTS does not require extensive training or the construction of large-scale datasets for fine-tuning.

9 Conclusion

We present EfficientUICoder, a bidirectional compression framework for efficient UI code generation. We identify the substantial redundancies in both image and code tokens that not only increase computational overhead but also hinder model focus on key UI elements. To address these issues, EfficientUICoder integrates three complementary techniques: (i) Element and Layout-aware Token Compression, which condenses visual inputs while preserving essential UI structures; (ii) Region-aware Token Refinement, which selectively enhances semantically important tokens; and (iii) Adaptive Duplicate Token Suppression, which dynamically penalizes repetitive code generation. Experimental results demonstrate that EfficientUICoder achieves a 55%–60% compression ratio without sacrificing quality, while significantly reducing prefill time, FLOPS, and inference latency.

Data Availability

All the code for EfficientUICoder are available at https://github.com/WebPAI/EfficientUICoder.

Acknowledgment

The work was supported by three grants from the Research Grants Council of the Hong Kong Special Administrative Region, China: (1) No. SRFS2425-4S03 of the Senior Research Fellow Scheme, (2) No. CUHK 14209124 of the General Research Fund and (3) No. PF23-87650 of Jingyu Xiao's Hong Kong PhD Fellowship Scheme. This research was also supported by the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grant (Project ID: 23-SIS-SMU-088). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

[1] Batuhan Aşıroğlu, Büşta Rümeysa Mete, Eyyüp Yıldız, Yağız Nalçakan, Alper Sezen, Mustafa Dağtekin, and Tolga Ensari. 2019. Automatic HTML code generation from mock-up images using machine learning techniques. In *2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT)*. Ieee, 1–4. doi:10.1109/EBBT.2019.8741736

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025). doi:10.48550/arXiv.2502.13923

[3] Tony Beltramelli. 2018. pix2code: Generating Code from a Graphical User Interface Screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems* (Paris, France) *(EICS '18)*. Association for Computing Machinery, New York, NY, USA, Article 3, 6 pages. doi:10.1145/3220134.3220135

[4] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXXXI* (Milan, Italy). Springer-Verlag, Berlin, Heidelberg, 19–35. doi:10.1007/978-3-031-73004-7_2

[5] Wen-Yin Chen, Pavol Podstreleny, Wen-Huang Cheng, Yung-Yao Chen, and Kai-Lung Hua. 2022. Code generation from a graphical user interface via attention-based encoder–decoder model. *Multimedia Systems* 28, 1 (2022), 121–130. doi:10.1007/s00530-021-00804-7

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020). doi:10.48550/arXiv.2010.11929

[7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[8] Zihao Fu, Wai Lam, Anthony Man-Cho So, and Bei Shi. 2021. A theoretical analysis of the repetition problem in text generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12848–12856. doi:10.48550/arXiv.2012.14660

[9] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, Wenbin Jiang, Hai Jin, and Xiangliang Zhang. 2025. WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1834–1845. doi:10.1145/3696410.3714889

[10] Yi Gui, Zhen Li, Zhongyi Zhang, Guohao Wang, Tianpeng Lv, Gaoyang Jiang, Yi Liu, Dongping Chen, Yao Wan, Hongyu Zhang, Wenbin Jiang, Xuanhua Shi, and Hai Jin. 2025. LaTCoder: Converting Webpage Design to Code with Layout-as-Thought. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 721–732. doi:10.1145/3711896.3737016

[11] Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang, and Xiangliang Zhang. 2025. UICopilot: Automating UI Synthesis via Hierarchical Code Generation from Webpage Designs *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1846–1855. doi:10.1145/3696410.3714891

[12] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to Stop? Towards Efficient Code Generation in LLMs with Excess Token Prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1073–1085. doi:10.1145/3650212.3680343

[13] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751* (2019). doi:10.48550/arXiv.1904.09751

[14] Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, and Shashwat Gulyani. 2019. Sketch2Code: transformation of sketches to UI in real-time using deep neural network. *arXiv preprint arXiv:1910.08930* (2019). doi:10.48550/arXiv.1910.08930

[15] Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. arXiv:2403.09029 [cs.HC] doi:10.48550/arXiv.2403.09029

[16] Huayang Li, Tian Lan, Zihao Fu, Deng Cai, Lemao Liu, Nigel Collier, Taro Watanabe, and Yixuan Su. 2023. Repetition in repetition out: towards understanding neural text degeneration from the data perspective. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NeurIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 3187, 16 pages. doi:10.48550/arXiv.2310.10226

[17] Haiming Li, Qiyang Xia, Yong Wang, et al. 2017. Research and improvement of kruskal algorithm. *Journal of Computer and Communications* 5, 12 (2017), 63. doi:10.4236/jcc.2017.512007

[18] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*. doi:10.48550/arXiv.2304.08485

[19] Tuan Anh Nguyen and Christoph Csallner. 2015. Reverse engineering mobile application user interfaces with REMAUI *(ASE '15)*. IEEE Press, 248–259. doi:10.1109/ASE.2015.32

[20] Dangfeng Pan, Zhensu Sun, Cenyuan Zhang, David Lo, and Xiaoning Du. 2025. The Hidden Cost of Readability: How Code Formatting Silently Consumes Your LLM Budget. *2026 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (2025). doi:10.48550/arXiv.2508.13666

[21] Wei Pang, Kevin Qinghong Lin, Xiangru Jian, Xi He, and Philip Torr. 2026. Paper2Poster: Towards Multimodal Poster Automation from Scientific Papers. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. doi:10.48550/arXiv.2505.21497

[22] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Pierre Isabelle, Eugene Charniak, and Dekang Lin (Eds.). Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318. doi:10.3115/1073083.1073135

[23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763. doi:10.48550/arXiv.2103.00020

[24] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol (TOSEM)*. 34, 5, Article 137 (May 2025), 22 pages. doi:10.1145/3708525

[25] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3956–3974. doi:10.18653/v1/2025.naacl-long.199

[26] Zhensu Sun, Chengran Yang, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2025. Token Sugar: Making Source Code Sweeter for LLMs through Token-Efficient Shorthand. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, 2440–2451. doi:10.1109/ASE63991.2025.00201

[27] Wenxin Tang, Jingyu Xiao, Wenxuan Jiang, Xi Xiao, Yuhang Wang, Xuxin Tang, Qing Li, Yuehe Ma, Junliang Liu, Shisong Tang, and Michael R. Lyu. 2025. SlideCoder: Layout-aware RAG-enhanced Hierarchical Slide Generation from Design. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 9015–9039. doi:10.18653/v1/2025.emnlp-main.458

[28] Yuxuan Wan, Yi Dong, Jingyu Xiao, Yintong Huo, Wenxuan Wang, and Michael R Lyu. 2024. MRWeb: An Exploration of Generating Multi-Page Resource-Aware Web Code from UI Designs. *arXiv preprint arXiv:2412.15310* (2024). doi:10.48550/arXiv.2412.15310

[29] Yuxuan Wan, Tingshuo Liang, Jiakai Xu, Jingyu Xiao, Yintong Huo, and Michael R Lyu. 2025. Automatically Generating Web Applications from Requirements Via Multi-Agent Test-Driven Development. *arXiv preprint arXiv:2509.25297* (2025). doi:10.48550/arXiv.2509.25297

[30] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael Lyu. 2025. Divide-and-Conquer: Generating UI Code from Screenshots. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE094 (June 2025), 24 pages. doi:10.1145/3729364

[31] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. arXiv:1910.03771 [cs.CL] doi:10.48550/arXiv.1910.03771

[32] Fan Wu, Cuiyun Gao, Shuqing Li, Xin-Cheng Wen, and Qing Liao. 2025. MLLM-Based UI2Code Automation Guided by UI Layout Information. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA050 (June 2025), 23 pages. doi:10.1145/3728925

[33] Jingyu Xiao, Jiantong Qin, Shuoqi Li, Man Ho Lam, Yuxuan Wan, Jen-tse Huang, Yintong Huo, and Michael R Lyu. 2026. ComUICoder: Component-based Reusable UI Code Generation for Complex Websites via Semantic Segmentation and Element-wise Feedback. *arXiv preprint arXiv:2602.19276* (2026). doi:10.48550/arXiv.2602.19276

[34] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R. Lyu. 2025. Interaction2Code: Benchmarking MLLM-based Interactive Webpage Code Generation from Interactive Prototyping. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, 241–253. doi:10.1109/ASE63991.2025.00028

[35] Jingyu Xiao, Ming Wang, Man Ho Lam, Yuxuan Wan, Junliang Liu, Yintong Huo, and Michael R Lyu. 2025. Designbench: A comprehensive benchmark for mllm-based front-end code generation. *arXiv preprint arXiv:2506.06251* (2025). doi:10.48550/arXiv.2506.06251

[36] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) *(ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1655–1659. doi:10.1145/3368089.3417940

[37] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. 2024. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247* (2024). doi:10.48550/arXiv.2410.17247

[38] Mikio Yamamoto and Kenneth W Church. 2001. Using suffix arrays to compute term frequency and document frequency for all substrings in a corpus. *Computational Linguistics* 27, 1 (2001), 1–30. doi:10.1162/089120101300346787

[39] Guang Yang, Yu Zhou, Wei Cheng, Xiangyu Zhang, Xiang Chen, Terry Yue Zhuo, Ke Liu, Xin Zhou, David Lo, and Taolue Chen. 2026. Less Is More: DocString Compression in Code Generation. *ACM Trans. Softw. Eng. Methodol (TOSEM)*. 35, 2, Article 41 (Jan. 2026), 31 pages. doi:10.1145/3735636

[40] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. 2025. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*. 19792–19802. doi:10.48550/arXiv.2412.04467

[41] Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Qazim Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, Haonan Li, Preslav Nakov, Timothy Baldwin, Zhengzhong Liu, Eric P. Xing, Xiaodan Liang, and Zhiqiang Shen. 2024. Web2Code: a large-scale webpage-to-code dataset and evaluation framework for multimodal LLMs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NeurIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 3560, 24 pages. doi:10.48550/arXiv.2406.20098

[42] Sai Zhang, Zhenchang Xing, Ronghui Guo, Fangzhou Xu, Lei Chen, Zhaoyuan Zhang, Xiaowang Zhang, Zhiyong Feng, and Zhiqiang Zhuang. 2025. Empowering Agile-Based Generative Software Development through Human-AI Teamwork. *ACM Trans. Softw. Eng. Methodol (TOSEM)*. 34, 6, Article 156 (July 2025), 46 pages. doi:10.1145/3702987

[43] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. 2025. SparseVLM: visual token sparsification for efficient vision-language model inference. In *Proceedings of the 42nd International Conference on Machine Learning* (Vancouver, Canada) *(ICML'25)*. JMLR.org, Article 3000, 18 pages. doi:10.48550/arXiv.2410.04417

[44] Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. 2025. DeclarUI: Bridging Design and Development with Automated Declarative UI Code Generation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE011 (June 2025), 23 pages. doi:10.1145/3715726

Received 2026-02-24; accepted 2026-03-24